

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

TEMA:

Análisis crítico de CockroachDB como sistema distribuido: transparencia, modelo CAP, comunicación y tolerancia a fallos

ACTIVIDAD:

TA-IND-01 — Informe Académico Individual — Análisis Crítico de Sistemas Distribuidos

Estudiante: Jeremy Ruperto Gaibor Rodríguez

Docente: Guerrero Ulloa Gleiston Cicerón

Asignatura: Aplicaciones Distribuidas

Curso: Séptimo Nivel — Paralelo A

Período académico: 2025–2026

Fecha: 20 de mayo de 2026

Resumen

Este informe analiza CockroachDB como caso concreto de sistema distribuido, con foco en transparencia de distribución, modelo CAP, comunicación entre nodos y tolerancia a fallos. El análisis no se limita a describir la herramienta, sino que identifica decisiones de diseño que pueden aplicarse al PFC del equipo, orientado a la venta de componentes de computadora y armado de PC bajo pedido. La conclusión central es que el PFC no necesita replicar una arquitectura geodistribuida como CockroachDB, pero sí debe aplicar principios de consistencia, separación de módulos, respaldo de datos críticos y recuperación ante fallos [2, 5].

Palabras clave: CockroachDB, sistema distribuido, CAP, tolerancia a fallos, base de datos distribuida.

Índice

Resumen	
1 Introducción	3
2 Fundamentación teórica	3
3 Metodología de análisis	4
4 Descripción de CockroachDB	4
5 Análisis crítico	5
5.1 Transparencia de distribución	5
5.2 Modelo CAP aplicado a CockroachDB	5
5.3 Comunicación entre componentes	6
5.4 Tolerancia a fallos	6
6 Aplicación al PFC del equipo	6
7 Conclusiones	7
8 Repositorio	7
Referencias	8

1. Introducción

Diseñar un sistema distribuido implica decidir qué se prioriza: consistencia, disponibilidad, rendimiento o tolerancia a fallos. Cada elección afecta el comportamiento del sistema bajo carga o ante fallos. Por eso, analizar un caso real permite observar cómo los principios teóricos se convierten en decisiones de arquitectura [1, 6].

CockroachDB es una base de datos SQL distribuida diseñada para transacciones consistentes sobre hardware común. Su arquitectura combina SQL, replicación, rangos de claves y coordinación entre nodos para mantener la integridad de los datos en entornos distribuidos [2, 5].

El informe examina cómo CockroachDB aplica los conceptos de la Unidad 1: transparencia, modelo CAP, comunicación y tolerancia a fallos. Además, relaciona esos conceptos con el PFC del equipo: un sistema de venta de componentes de computadora y armado de PC bajo pedido, donde el inventario, los pagos y los pedidos requieren consistencia y continuidad operativa [2, 3].

2. Fundamentación teórica

Un sistema distribuido está compuesto por nodos independientes que se comunican mediante red para operar como una sola unidad lógica. La ausencia de memoria compartida obliga a usar comunicación por mensajes, coordinación y mecanismos de control para mantener coherencia entre componentes [1].

La transparencia de distribución consiste en ocultar al usuario ciertos detalles internos del sistema, como la ubicación de los recursos, la replicación o el acceso remoto. Esta propiedad es útil porque permite usar el sistema como una unidad lógica, aunque internamente esté distribuido. Sin embargo, la transparencia no elimina por completo efectos como latencia, fallos parciales o errores de comunicación [1].

El modelo CAP establece compromisos entre consistencia, disponibilidad y tolerancia a particiones. Brewer aclara que la idea de “dos de tres” simplifica demasiado el problema, porque los sistemas reales ajustan sus decisiones según el manejo de particiones, la recuperación y el contexto de uso [6].

La tolerancia a fallos es la capacidad de mantener el servicio ante fallos parciales. En sistemas distribuidos, esta propiedad suele apoyarse en replicación, detección de fallos, recuperación y mecanismos de consenso. Aun así, tolerar fallos implica costos de coordinación, rendimiento y complejidad operativa [7].

3. Metodología de análisis

El informe se desarrolló como un análisis documental y crítico de un sistema distribuido real. Se seleccionó CockroachDB porque permite estudiar de forma directa conceptos de la Unidad 1: transparencia de distribución, modelo CAP, comunicación entre nodos y tolerancia a fallos.

La revisión se basó en fuentes académicas y técnicas verificables, priorizando trabajos recientes sobre CockroachDB, bases NewSQL, sistemas multirregión y tolerancia a fallos. También se utilizaron fuentes teóricas para fundamentar conceptos generales de sistemas distribuidos y CAP.

El análisis se organizó en tres pasos: primero, identificación de los conceptos teóricos; segundo, descripción técnica de CockroachDB como caso de estudio; y tercero, relación crítica con el PFC del equipo, orientado a la venta de componentes de computadora y armado de PC bajo pedido.

4. Descripción de CockroachDB

CockroachDB es una base de datos SQL geodistribuida diseñada para ejecutar transacciones consistentes sobre clústeres de hardware común. Taft et al. la describen como una base que combina una interfaz SQL con una arquitectura de almacenamiento distribuido basada en rangos de claves replicados [2].

Internamente, los datos se organizan en rangos que se replican entre nodos del clúster. Cada rango usa Raft para coordinar escrituras y mantener consistencia entre réplicas. Esta organización evita depender de un coordinador único para todo el clúster, ya que la coordinación ocurre a nivel de rango [2, 5].

En entornos multirregión, CockroachDB permite configurar localidad de datos y disponibilidad mediante instrucciones SQL. Esto facilita ubicar datos críticos cerca de ciertos usuarios o regiones sin administrar manualmente toda la replicación desde la infraestructura [3, 4].

5. Análisis crítico

5.1. Transparencia de distribución

CockroachDB ofrece transparencia de acceso y ubicación: el usuario ejecuta SQL sin manejar directamente nodos, rangos ni réplicas. Esta abstracción reduce la complejidad para el desarrollador, porque la base se presenta como una sola unidad lógica aunque internamente esté distribuida [1, 2].

La transparencia tiene límites. En escenarios multirregión, una escritura puede requerir coordinación entre réplicas ubicadas en regiones diferentes. Esa coordinación no se ve en la interfaz SQL, pero sí puede reflejarse en mayor latencia durante operaciones críticas como reservar inventario o confirmar pagos [3, 4].

Otro límite aparece ante fallos. Si las réplicas necesarias no alcanzan mayoría, las escrituras se detienen para evitar inconsistencias. Esto protege los datos, pero obliga a que la aplicación maneje errores y defina qué hacer cuando una operación no puede completarse [2, 7].

5.2. Modelo CAP aplicado a CockroachDB

CockroachDB prioriza consistencia y tolerancia a particiones en operaciones transaccionales. Sus escrituras requieren coordinación entre réplicas antes de considerarse confirmadas. Si no se alcanza mayoría, el sistema prefiere no aceptar la escritura antes que registrar datos inconsistentes [2, 6].

En datos críticos, esta decisión tiene sentido. En sistemas de inventario, pagos o pedidos, aceptar escrituras sin coordinación puede producir errores difíciles de corregir. Por ejemplo, vender dos veces el mismo componente cuando solo existe una unidad afectaría directamente la operación del negocio [2, 5].

El costo de esta decisión está en la latencia y la disponibilidad. Mantener consistencia fuerte requiere coordinación, y esa coordinación puede afectar el tiempo de respuesta cuando los datos están distribuidos. Por eso, CockroachDB no elimina los compromisos de CAP; los gestiona priorizando integridad transaccional [3, 6].

5.3. Comunicación entre componentes

La comunicación en CockroachDB sigue un modelo por capas: el cliente se conecta a un nodo SQL, y ese nodo coordina con los nodos responsables de los rangos afectados. Si una transacción involucra varios rangos, puede requerir comunicación con distintos nodos antes de confirmarse [2, 3].

Este diseño permite distribuir la carga y evitar un coordinador global único, pero aumenta la importancia del diseño de datos. Si las operaciones frecuentes cruzan muchos rangos o regiones, el rendimiento puede verse afectado. Por eso, la ubicación de datos y la organización de claves no son detalles secundarios [3, 5].

En versiones recientes, CockroachDB también ha evolucionado hacia escenarios serverless y multirregión, separando componentes y usando virtualización de clústeres para mejorar escalabilidad operativa. Esto refuerza la importancia de la comunicación interna y de la ubicación de datos como decisiones centrales en su arquitectura [4].

5.4. Tolerancia a fallos

CockroachDB tolera fallos mediante replicación y consenso. Mientras exista una mayoría de réplicas disponibles, el sistema puede continuar operando sin depender de un único servidor principal. Esta propiedad lo diferencia de una arquitectura centralizada sin replicación [2, 7].

Sin embargo, tolerar fallos no significa tolerar cualquier escenario. Si una partición de red deja a un grupo sin mayoría, ese grupo no puede confirmar escrituras. Esta decisión evita inconsistencias, pero reduce la disponibilidad de las operaciones afectadas hasta que se recupere la conectividad [2, 6].

La ubicación física también importa. Si las réplicas están concentradas en una sola zona, un fallo de esa zona puede afectar el servicio. Distribuir réplicas entre regiones mejora la tolerancia a fallos geográficos, pero puede aumentar la latencia. No existe una configuración única óptima; depende de las prioridades del sistema [3, 6].

6. Aplicación al PFC del equipo

El PFC es un sistema de venta de componentes de computadora y armado de PC bajo pedido. Los datos más críticos son inventario, pedidos, pagos y estado del armado. El análisis de CockroachDB permite identificar decisiones concretas para evitar errores en esos datos [2, 5].

Primera decisión: separar módulos por nivel de consistencia. El inventario y los pagos necesitan consistencia estricta, porque vender un componente inexistente o registrar un pago duplicado afecta directamente el negocio. En cambio, el catálogo o el historial de navegación pueden tolerar información ligeramente desactualizada [1, 6].

Segunda decisión: definir qué ocurre si el sistema falla durante una operación crítica. En el flujo de armado de PC, el sistema debe consultar inventario, reservar componentes, registrar el pedido y confirmar el pago. Si un paso falla, pueden quedar estados incompletos; por eso, los estados del pedido deben ser claros y recuperables [2, 7].

Tercera decisión: evitar un punto único de falla en los datos críticos. El PFC no necesita un clúster geodistribuido como CockroachDB, pero sí requiere una estrategia mínima de continuidad: respaldos automáticos, control de transacciones y procedimiento de recuperación documentado [5, 7].

7. Conclusiones

El análisis de CockroachDB permite observar que un sistema distribuido no se define solo por usar varios nodos, sino por las decisiones que toma frente a consistencia, disponibilidad, comunicación y fallos. En este caso, la arquitectura prioriza la integridad de los datos, aunque eso implique costos de coordinación y latencia.

La distribución no resuelve automáticamente los problemas de una aplicación. Al contrario, obliga a decidir qué datos requieren mayor protección, qué operaciones pueden esperar y qué errores deben manejarse de forma explícita.

Para el PFC, CockroachDB funciona como un caso de referencia para identificar decisiones importantes, no como un modelo que deba replicarse. El proyecto puede ser más simple, pero debe conservar una idea central: los datos críticos necesitan reglas claras de consistencia, respaldo y recuperación para que el sistema siga siendo confiable cuando ocurra un fallo.

8. Repositorio

[https://github.com/JeremyGaibor/Aplicaciones-Distribuidas/tree/main/Tema%3A%20TA-IND-01%20%E2%80%94%20Informe%20Acad%C3%A9mico%20Individual%20%E2%80%94%20An%C3%A1lisis%20Cr%C3%ADtico%20de%20Sistemas%20Distribuidos%20\(Unidad%201\)](https://github.com/JeremyGaibor/Aplicaciones-Distribuidas/tree/main/Tema%3A%20TA-IND-01%20%E2%80%94%20Informe%20Acad%C3%A9mico%20Individual%20%E2%80%94%20An%C3%A1lisis%20Cr%C3%ADtico%20de%20Sistemas%20Distribuidos%20(Unidad%201))

Referencias

- [1] M. van Steen and A. S. Tanenbaum, *Distributed Systems*, 4th ed. Leiden, The Netherlands: Maarten van Steen, 2023. [Online]. Available: <https://www.distributed-systems.net/>
- [2] R. Taft et al., “CockroachDB: The Resilient Geo-Distributed SQL Database,” in *Proc. 2020 ACM SIGMOD Int. Conf. Management of Data*, 2020, pp. 1493–1509, doi: 10.1145/3318464.3386134.
- [3] A. Ajmani et al., “Enabling the Next Generation of Multi-Region Applications with CockroachDB,” in *Proc. 2022 Int. Conf. Management of Data*, 2022, doi: 10.1145/3514221.3526053.
- [4] J. Swenson et al., “CockroachDB Serverless: Sub-second Scaling from Zero with Multi-region Cluster Virtualization,” in *Proc. Companion of the 2025 Int. Conf. Management of Data*, 2025, doi: 10.1145/3722212.3724432.
- [5] E. Pina, F. Sá, and J. Bernardino, “NewSQL Databases Assessment: CockroachDB, MariaDB Xpand, and VoltDB,” *Future Internet*, vol. 15, no. 1, Art. no. 10, 2023, doi: 10.3390/fi15010010.
- [6] E. Brewer, “CAP Twelve Years Later: How the ‘Rules’ Have Changed,” *Computer*, vol. 45, no. 2, pp. 23–29, 2012, doi: 10.1109/MC.2012.37.
- [7] S. Isukapalli and S. N. Srirama, “A systematic survey on fault-tolerant solutions for distributed data analytics: Taxonomy, comparison, and future directions,” *Computer Science Review*, vol. 53, Art. no. 100660, 2024, doi: 10.1016/j.cosrev.2024.100660.